

TRABAJO PRÁCTICO N.º 1 – AUTÓMATAS

UNIVERSIDAD TECNOLÓGICA NACIONAL – FRBA

Sintaxis y Semántica de los Lenguajes

Integrantes:

Nicole Brunstein

Ariana Castro

Máximo Colombatto

Federico Dimentstein

Agustina Marques Serra

Curso: K2002

Docente: Ing. Roxana Leituz

Ejercicio 1

Decisiones tomadas

Para la resolución de este ejercicio del TP tuvimos que tomar ciertas decisiones sobre algunos casos que no estaban especificados en la consigna. A continuación, listamos las decisiones tomadas:

- Notación octal: decidimos representar los octales con uno o más ceros iniciales y al menos un dígito entre 1 y 7. Ej.: 07, 012, 077.
- Notación hexadecimal: decidimos usar el prefijo 0x o 0X. Ej.: 0xAF, 0X25.
- Números formados únicamente por ceros: decidimos considerarlos decimales y no octales. Ej.: 0, 00, 000.....
- Letras hexadecimales: decidimos aceptar tanto mayúsculas como minúsculas: A..F y a..f.
- 08 y 09: como un número que comienza con 0 intenta reconocerse como octal, decidimos que estos casos sean considerados error léxico, en lugar de interpretarlos como decimales.
- 0x sin dígitos posteriores: decidimos que sea inválido. Después de 0x o 0X debe aparecer al menos un dígito hexadecimal.
- Separador final: decidimos que una cadena como 12@07@ sea inválida, ya que interpretamos @ como separador entre dos constantes y, por lo tanto, después de él debe comenzar otra.

Implementación

Para implementar el autómata se utilizó una variable `estadoActual`, que indica el estado en el que se encuentra el autómata durante el recorrido de la cadena.

Antes de comenzar el reconocimiento, se verifica que todos los caracteres de la cadena pertenezcan al alfabeto definido. Si aparece un carácter no válido, se informa el error y no se continúa con el procesamiento.

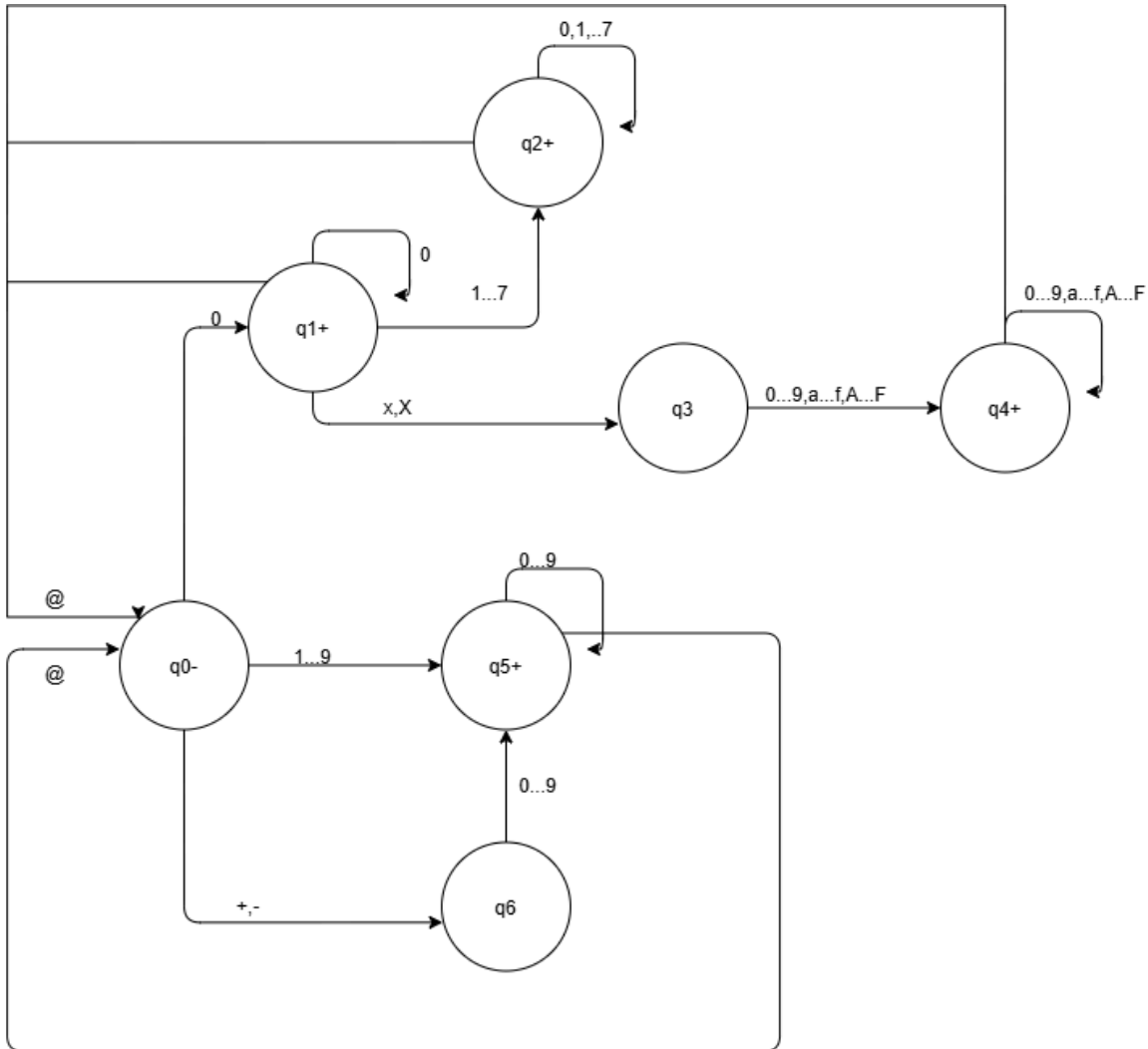
La cadena se recorre carácter por carácter. Según el estado actual y el carácter leído, se obtiene el siguiente estado utilizando la tabla de transiciones.

Cuando se encuentra el separador @, si el estado actual es un estado final, se contabiliza la constante reconocida y, mediante la transición correspondiente, se vuelve al estado inicial `q0` para analizar la siguiente constante. Si @ aparece en un estado no válido para finalizar una constante, la transición conduce al estado de error `q7`.

Si durante el recorrido una transición conduce al estado `q7`, la cadena se considera inválida y se informa que no pertenece al lenguaje.

Al finalizar la cadena, se verifica que el autómata haya quedado en un estado final. En ese caso, se contabiliza también la última constante reconocida.

Autómata



Definición formal

El autómata se define formalmente como:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Donde:

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$ es el conjunto de estados.
- $\Sigma = \{0..9, a..f, A..F, x, X, +, -, @\}$ es el alfabeto utilizado.
- δ es la función de transición, definida en la tabla de transiciones presentada a continuación.
- q_0 es el estado inicial.
- $F = \{q_1, q_2, q_4, q_5\}$ es el conjunto de estados finales.
- q_7 estado de rechazo o estado trampa.

Tabla de transiciones

Estado	+, -	0	1..7	8, 9	x, X	a..f, A..F	@
q0	q6	q1	q5	q5	q7	q7	q7
q1	q7	q1	q2	q7	q3	q7	q0
q2	q7	q2	q2	q7	q7	q7	q0

q3	q7	q4	q4	q4	q7	q4	q7
q4	q7	q4	q4	q4	q7	q4	q0
q5	q7	q5	q5	q5	q7	q7	q0
q6	q7	q5	q5	q5	q7	q7	q7
q7	q7	q7	q7	q7	q7	q7	q7

Significado de los estados

- q0: estado inicial, también se vuelve a él luego de reconocer un número y leer @.
- q1: se leyó uno o más 0. Si el número termina ahí, se considera decimal. También puede ser el comienzo de un octal o hexadecimal.
- q2: se reconoció una constante octal.
- q3: se leyó el prefijo 0x o 0X, pero todavía falta al menos un dígito hexadecimal.
- q4: se reconoció una constante hexadecimal.
- q5: se reconoció una constante decimal.
- q6: se leyó un signo + o - y se espera un dígito decimal.
- q7: estado de error o rechazo.

Los números formados únicamente por ceros (0, 00, 000, etc.) son considerados decimales. Cuando luego de uno o más ceros aparece un dígito entre 1 y 7, el número pasa a reconocerse como una constante octal. De esta forma, el cero inicial actúa como indicador de una posible constante octal.

Casos de prueba

Para comprobar el funcionamiento del autómata se probaron cadenas válidas e inválidas.

Entrada	Resultado esperado	Motivo
123@077@0xAF@-45	2 decimales, 1 octal, 1 hexadecimal	Cadena válida con los tres tipos
0@07@0xA@+80	2 decimales, 1 octal, 1 hexadecimal	Verifica el 0 decimal y el signo +
+8@012@0X25	1 decimal, 1 octal, 1 hexadecimal	Verifica hexadecimal con X mayúscula
123@078@45	Cadena inválida	8 no es un dígito octal
123@0xAG@45	Cadena inválida	G no pertenece a los dígitos hexadecimales
123@-@45	Cadena inválida	Después del signo debe aparecer un dígito
123@0x	Cadena inválida	Falta un dígito hexadecimal después de 0x
12@07@	Cadena inválida	La cadena termina con un separador

Capturas de las pruebas

```
● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ ./ejercicio1/ejercicio1
Ingrese la cadena: 123@077@0xAF@-45
Decimales: 2
Octales: 1
Hexadecimales: 1
● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ █

● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ ./ejercicio1/ejercicio1
Ingrese la cadena: 123@078@45
Error lexico.
● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ █

● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ ./ejercicio1/ejercicio1
Ingrese la cadena: 123@0xAG@45
Error lexico.
● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ █

● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ ./ejercicio1/ejercicio1
Ingrese la cadena: 123@0x
Error lexico.
● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ █

● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ ./ejercicio1/ejercicio1
Ingrese la cadena: 123@-
Error lexico.
● nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ █
```

Ejercicio 2

Implementación

Para realizar la conversión se creó la siguiente función:

```
int caracterAEntero(char c) {
    return c - '0';
}
```

En C, un carácter numérico como '7' no es lo mismo que el número entero 7.

Los caracteres numéricos '0' a '9' tienen valores consecutivos en la codificación de caracteres utilizada por C. Por este motivo, al restar '0' se obtiene el valor entero correspondiente.

Por ejemplo:

'7' - '0' = 7

De esta manera se puede convertir cualquier carácter comprendido entre '0' y '9' a su correspondiente número entero.

Validación de la entrada

Antes de realizar la conversión verificamos que el carácter ingresado se encuentre entre '0' y '9'.

Para esto utilizamos la siguiente condición:

```
caracter < '0' || caracter > '9'
```

Si la condición se cumple significa que el carácter ingresado no es numérico.
Mediante un while se vuelve a solicitar un carácter hasta que el usuario ingrese uno válido:

```
while (caracter < '0' || caracter > '9') {  
    printf("Error: debe ingresar un caracter numerico.\n");  
    printf("Ingrese un caracter numerico: ");  
    scanf(" %c", &caracter);  
}
```

Una vez que el carácter es válido, se llama a la función `caracterAEntero` y se muestra el resultado.

Casos de prueba

Para comprobar el funcionamiento del programa se realizaron pruebas con caracteres válidos e inválidos.

Entrada	Resultado esperado
7	El número entero es 7
0	El número entero es 0
9	El número entero es 9
a	Se informa error y se vuelve a pedir el carácter
#	Se informa error y se vuelve a pedir el carácter

Capturas de las pruebas

Ingreso válido

```
nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ ./ejercicio2/ejercicio2  
Ingrese un caracter numerico: 7  
El numero entero es: 7  
nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$
```

Ingreso inválido

```
nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$ ./ejercicio2/ejercicio2  
Ingrese un caracter numerico: a  
Error: debe ingresar un caracter numerico.  
Ingrese un caracter numerico: 2  
El numero entero es: 2  
nicole@WINDOWS-UCB1LK8:/mnt/c/Users/Nicole/Documents/TRABAJO-PRACTICO-1-AUTOMATAS-$
```

Ejercicio 3

Decisiones tomadas

Para la resolución del ejercicio se decidió dividir el problema en tres etapas:

1. Verificar que todos los caracteres ingresados pertenezcan al alfabeto permitido.
2. Validar mediante un autómata finito determinista que la cadena represente una operación correctamente formada.
3. Evaluar la expresión respetando la precedencia de los operadores.

Se aceptan únicamente:

- números enteros decimales formados por uno o más dígitos entre 0 y 9;
- los operadores +, - y *.

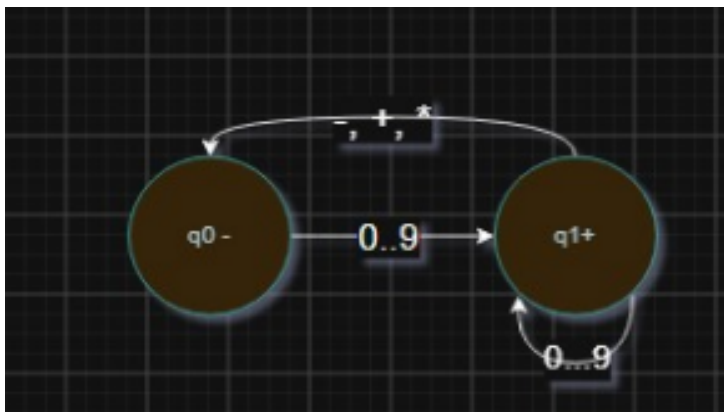
La expresión debe comenzar con un número y, después de cada operador, debe aparecer obligatoriamente otro número.

Por lo tanto, se consideran inválidas las expresiones que:

- comiencen con un operador;
- terminen con un operador;
- contengan dos operadores consecutivos;
- contengan caracteres que no pertenezcan al alfabeto definido.

Para realizar la operación se decidió dar mayor precedencia al operador * respecto de los operadores + y -.

Autómata



El autómata utiliza los siguientes estados:

- q0: estado inicial. Se espera el comienzo de un número. Este mismo estado se utiliza luego de reconocer un operador, ya que en ambos casos el símbolo siguiente debe ser obligatoriamente un dígito.
- q1: se reconoció al menos un dígito. Desde este estado puede continuar otro dígito o aparecer un operador. Es el único estado de aceptación.
- q2: estado de rechazo o estado trampa. Se alcanza cuando aparece una secuencia que no corresponde a una expresión válida.

Una vez alcanzado q2, cualquier símbolo posterior mantiene al autómata en dicho estado y la cadena es rechazada.

Definición formal

El autómata utilizado para reconocer las expresiones se define formalmente como:

$$M = (Q, \Sigma, \delta, q_0, F)$$

donde:

- $Q = \{q_0, q_1, q_2\}$ es el conjunto de estados.

- $\Sigma = \{0..9, +, -, * \}$ es el alfabeto.
- δ es la función de transición, definida mediante la tabla de transiciones.
- q_0 es el estado inicial.
- $F = \{q_1\}$ es el conjunto de estados finales.
- q_2 es el estado de rechazo o estado trampa.

Tabla de transiciones

Estado / Entrada	0..9	+, -, *
q0	q1	q2
q1	q1	q0
q2	q2	q2

El estado q_1 es el único estado de aceptación, ya que una expresión válida debe finalizar necesariamente con un número.

El estado q_2 representa un estado de rechazo o estado trampa. Toda entrada inválida conduce a este estado y, una vez alcanzado, la cadena no puede volver a ser aceptada.

Implementación

La solución se dividió en distintas funciones, cada una con una responsabilidad específica.

Clasificación de caracteres

La función `columna()` clasifica cada carácter de entrada según corresponda a:

- un operador (+, - o *);
- un dígito (0..9);
- un carácter no perteneciente al alfabeto.

Esta clasificación permite determinar qué columna de la tabla de transiciones debe utilizarse.

Verificación del alfabeto

La función `verifica()` recorre la cadena y comprueba que todos los caracteres pertenezcan al alfabeto definido para el ejercicio.

Si se encuentra un carácter distinto de un dígito o de los operadores permitidos, la expresión se rechaza.

Por ejemplo:

2/3

es rechazada porque / no pertenece al alfabeto.

Validación mediante el autómata

La función `esPalabraLeng()` recorre la expresión carácter por carácter.

Para cada símbolo se obtiene su categoría mediante `columna()` y luego se consulta la tabla de transiciones para obtener el nuevo estado:

```
estadoActual = tt[estadoActual][columna(caracter)];
```

La expresión es aceptada únicamente si, al finalizar el recorrido, el autómata se encuentra en el estado final q_1 .

De esta manera, expresiones como:

2+3

son aceptadas, mientras que:

2++3

son rechazadas porque luego de reconocer el primer operador el autómata se encuentra en q_0 , donde se espera obligatoriamente un dígito.

Evaluación de la expresión

Una vez validada la expresión, se realiza la operación matemática.

Para ello se utilizan principalmente las funciones:

- `esOperacion()`: determina si un carácter es uno de los operadores permitidos.
- `procesarOperacion()`: procesa el operador anterior junto con el número actual y actualiza el término y el resultado acumulado.
- `evaluarExpresion()`: recorre la expresión completa y obtiene el resultado final.

Precedencia de operadores

Para respetar la precedencia matemática, el programa mantiene separados el resultado acumulado y el término que se está calculando.

Se utilizan principalmente las siguientes variables:

- `numeroActual`: almacena el número que se está leyendo.
- `terminoActual`: almacena el término que todavía puede ser afectado por una multiplicación.
- `resultadoTotal`: almacena los términos que ya fueron completados.
- `operadorAnterior`: almacena el operador que debe aplicarse al número actual.

Cuando se encuentra un operador o se llega al final de la cadena, se procesa `numeroActual` utilizando `operadorAnterior`.

- Si `operadorAnterior` es `*`, `numeroActual` se multiplica por `terminoActual`.
- Si es `+`, el término anterior se suma a `resultadoTotal` y `numeroActual` pasa a ser el nuevo término.
- Si es `-`, el término anterior se suma a `resultadoTotal` y se comienza un nuevo término con el valor negativo de `numeroActual`.

Por ejemplo, para la expresión $2+3*4$:

1. Se lee 2. Al encontrar `+`, se procesa el operador anterior (`+`): `terminoActual` pasa a valer 2.
2. Se lee 3. Al encontrar `*`, se procesa el operador anterior (`+`): `resultadoTotal` pasa a valer 2 y `terminoActual` pasa a valer 3.
3. Se lee 4. Al finalizar la cadena se procesa el operador anterior (`*`): $\text{terminoActual} = 3 * 4 = 12$.
4. Finalmente se retorna $\text{resultadoTotal} + \text{terminoActual}$, es decir $2 + 12 = 14$.

De esta forma, las multiplicaciones se mantienen dentro del término actual antes de incorporarlo al resultado total, respetando la precedencia de `*` sobre `+` y `-`.

Funcionamiento general

El funcionamiento del programa puede resumirse en los siguientes pasos:

1. Se obtiene la expresión a analizar.
2. Se verifica que todos sus caracteres pertenezcan al alfabeto.
3. Se utiliza el autómata para comprobar que la expresión esté correctamente formada.
4. Si la expresión es válida, se evalúa respetando la precedencia de operadores.
5. Finalmente se muestra el resultado obtenido.

Si la expresión contiene un carácter que no pertenece al alfabeto, se informa un error léxico.

Si los caracteres pertenecen al alfabeto pero la expresión no tiene una estructura válida, se informa que la operación está mal formada.

Casos de prueba

Entrada	Resultado esperado	Motivo
$3+4*7+3-5$	29	Se respeta la precedencia de *
$2+3*4$	14	Se realiza primero $3*4$
$2*3+4$	10	Se realiza primero $2*3$
$10-2*3$	4	Se realiza primero la multiplicación
$2*3*4+5$	29	Se permiten multiplicaciones encadenadas
123	123	Una expresión formada por un único número es válida
$1++2$	Error sintactico	No se permiten operadores consecutivos
2^*	Error sintactico	La expresión no puede terminar con un operador
$*2$	Error sintactico	La expresión no puede comenzar con un operador
$2/3$	Error léxico	/ no pertenece al alfabeto
$2+a$	Error léxico	a no pertenece al alfabeto
$2 + 3$	Error léxico	El espacio no pertenece al alfabeto

Capturas de las pruebas

Expresión válida.

```
colom@DESKTOP-7Q7PNL9 MINGW64 ~/OneDrive/Escritorio/TRABAJO-PRACTICO-1-AUTOMATAS-/ejercicio3 (main)
$ ./ejercicio3-alternativo.exe
Analizando expresion: 2+3*4
-----
ESTADO: EXPRESION VALIDA.
RESULTADO FINAL: 14
```

Error de sintaxis por operadores consecutivos.

```
colom@DESKTOP-7Q7PNL9 MINGW64 ~/OneDrive/Escritorio/TRABAJO-PRACTICO-1-AUTOMATAS-/ejercicio3 (main)
$ ./ejercicio3-alternativo.exe
Analizando expresion: 1++2
-----
ERROR SINTACTICO: LA OPERACION ESTA MAL FORMADA.
```

Error de sintaxis por terminar con un operador.

```
colom@DESKTOP-7Q7PNL9 MINGW64 ~/OneDrive/Escritorio/TRABAJO-PRACTICO-1-AUTOMATAS-/ejercicio3 (main)
$ ./ejercicio3-alternativo.exe
Analizando expresion: 2*
-----
ERROR SINTACTICO: LA OPERACION ESTA MAL FORMADA.
```

Error por carácter no válido.

```
colom@DESKTOP-7Q7PNL9 MINGW64 ~/OneDrive/Escritorio/TRABAJO-PRACTICO-1-AUTOMATAS-/ejercicio3 (main)
$ ./ejercicio3-alternativo.exe
Analizando expresion: 2/3
-----
ERROR LEXICO: HAY CARACTERES QUE NO PERTENECEN AL ALFABETO.
```

Instructivo de compilación y ejecución

- Para compilar y ejecutar los programas se debe contar con un compilador de lenguaje C, en este caso GCC.
- Los comandos indicados a continuación deben ejecutarse desde una terminal ubicada en la carpeta raíz del proyecto.

Ejercicio 1

Para compilar: `gcc ejercicio1/ejercicio1.c -o ejercicio1/ejercicio1`

Para ejecutar: `./ejercicio1/ejercicio1`

La cadena que se desea analizar se encuentra definida directamente en el código fuente mediante la variable:
`char palabra[] = "123@077@0xAF@+45";`

Para realizar una prueba con una cadena diferente, se debe modificar el contenido de la variable ``palabra`` y volver a compilar el programa.

Al ejecutarse, el programa realiza primero una verificación de los caracteres mediante la función ``verifica()``.

Esta función comprueba que todos los caracteres de la cadena pertenezcan al alfabeto considerado por el programa.

- Si se encuentra un carácter que no pertenece al alfabeto, se informa:
HAY CARACTERES QUE NO PERTENECEN AL ALFABETO

-Si todos los caracteres son válidos, la cadena se procesa mediante el autómata implementado con la tabla de transiciones ``tt``.

Durante el recorrido de la cadena se reconocen constantes decimales, octales y hexadecimales separadas por el carácter ``@``.

Si la cadena pertenece al lenguaje, el programa informa:

LA CADENA PERTENECE AL LENGUAJE y muestra la cantidad de constantes reconocidas de cada tipo:

Decimales: cantidad

Octales: cantidad

Hexadecimales: cantidad

Si la cadena utiliza caracteres válidos pero su estructura no corresponde al lenguaje reconocido por el autómata, se informa:

NO PERTENECE AL LENGUAJE

Ejercicio 2

- Para compilar: gcc ejercicio2/ejercicio2.c -o ejercicio2/ejercicio2
- Para ejecutar: ./ejercicio2/ejercicio2

Al ejecutarse, el programa solicita el ingreso de un carácter numérico mediante teclado.

Si el carácter ingresado se encuentra entre '0' y '9', se realiza la conversión a su correspondiente valor entero utilizando la función ``characterAEntero()`` y se muestra el resultado.

Si se ingresa un carácter que no se encuentra entre '0' y '9', el programa informa:

Error: debe ingresar un carácter numérico.

Luego vuelve a solicitar una entrada hasta que se ingrese un carácter numérico válido.

Ejercicio 3

- Para compilar: gcc ejercicio3/ejercicio3.c -o ejercicio3/ejercicio3
- Para ejecutar: ./ejercicio3/ejercicio3

La expresión que se desea analizar se encuentra definida dentro del código fuente mediante la variable:

```
char expresion[] = "2+";
```

Para realizar una prueba con una expresión diferente, se debe modificar el contenido de la variable ``expresion`` y volver a compilar el programa.

Al ejecutarse, el programa realiza el procesamiento en tres etapas.

En primer lugar, mediante la función ``verifica()``, se comprueba que todos los caracteres de la expresión pertenezcan al alfabeto definido, compuesto por los dígitos del 0 al 9 y los operadores ``+``, ``-`` y ``*``.

-Si aparece un carácter que no pertenece al alfabeto, se informa:

ERROR LÉXICO: HAY CARACTERES QUE NO PERTENECEN AL ALFABETO.

En segundo lugar, mediante la función ``esPalabraLeng()``, se utiliza el autómata finito para verificar que la expresión esté correctamente formada.

-Si todos los caracteres pertenecen al alfabeto pero la expresión no cumple con la estructura definida por el autómata, se informa:

ERROR SINTACTICO: LA OPERACIÓN ESTÁ MAL FORMADA.

-Finalmente, si la expresión supera ambas validaciones, se informa que es válida y se evalúa mediante la función ``evaluarExpresion()``.

Durante la evaluación se respeta la precedencia de la multiplicación sobre la suma y la resta.

El programa muestra finalmente el resultado obtenido mediante:

RESULTADO FINAL: valor